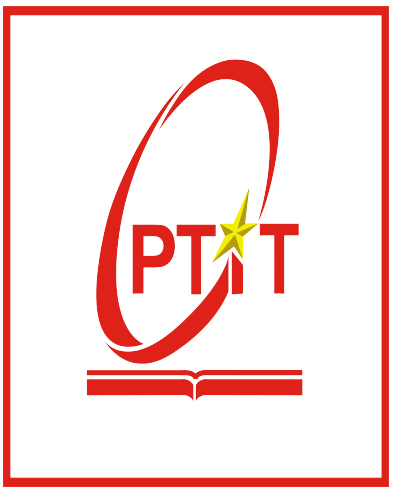
POSTS AND TELECOMMUNICATIONS
INSTITUTE OF TECHNOLOGY

FACULTY OF INFORMATION TECHNOLOGY I

PYTHON PROGRAMMING COURSE



ASSIGNMENT REPORT

Lecturer : KIM NGOC BACH

Student : DAO VAN KHANG - B23DCVT218

: VU MINH SANG - B23DCCE082

Class : D23CQCE04-B

Group : 04

Ha Noi – 2025

Summary

This report documents the implementation of Python Programming Assignment 2 for the 2024– 2025 academic year. The objective of the assignment is to perform image classification on the CIFAR-10 dataset by developing, training, and evaluating two distinct neural network architectures using the PyTorch library.

The task is divided into several key stages. The first stage involves model construction: a basic Multi-Layer Perceptron (MLP) neural network with 3 layers will be built, alongside a Convolutional Neural Network (CNN) featuring 3 convolution layers.

The second stage focuses on applying these models to image classification. This includes the complete pipeline of training, validating, and testing both the MLP and CNN on the CIFAR-10 data. As part of the evaluation, learning curves will be plotted to visualize training progress, and confusion matrices will be generated to assess classification performance for each model.

Finally, a comparative analysis will be conducted to discuss and contrast the results obtained from the two neural networks. The project submission will comprise this report in PDF format and the corresponding Python code, delivered via GitHub.

Contents

Chapter 1: Introduction

Image classification, a fundamental task in computer vision, involves assigning a label or class to an input image from a predefined set of categories. It forms the basis for numerous applications, ranging from autonomous driving and medical image analysis to content-based image retrieval and surveillance systems. The challenge lies in developing algorithms that can effectively learn and generalize visual patterns from raw pixel data to make accurate predictions on unseen images. Over the years, various machine learning approaches have been applied to this problem, with deep learning models, particularly neural networks, demonstrating state-of-the-art performance.

This project focuses on image classification using the CIFAR-10 dataset. The CIFAR-10 dataset (Canadian Institute For Advanced Research, 10 classes), accessible at https://www.cs.toronto.edu/~kriz/cifar.html, is a widely used benchmark collection for computer vision algorithms. It consists of 60,000 32×32 color images in 10 mutually exclusive classes, with 6,000 images per class. The dataset is pre-divided into 50,000 training images and 10,000 testing images, making it suitable for training and evaluating machine learning models. The ten classes represent airplanes, automobiles, birds, cats, deer, dogs, frogs, horses, ships, and trucks.

The primary tasks of this project include:

- Building a basic Multi-Layer Perceptron (MLP) neural network with 3 layers.
- Building a Convolutional Neural Network (CNN) with 3 convolution layers.
- Conducting image classification using both the MLP and CNN, which encompasses training, validation, and testing phases for each model.
- Visualizing the training progress and model performance by plotting learning curves

and confusion matrices.

- Performing a comparative analysis and discussion of the results obtained from the two neural network models.

Chapter 2: Methodology

This chapter details the procedures and techniques employed for the image classification task on the CIFAR-10 dataset. It covers dataset preparation, the architectures of the Multi-Layer Perceptron (MLP) and Convolutional Neural Network (CNN) models, the training process, and the metrics used for evaluation. All implementations were carried out using the PyTorch library.

2.1 Dataset Preparation

Dataset Source and Description :

- The CIFAR-10 dataset was obtained from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz.
- It contains 60,000 32×32 color images across 10 classes: `airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck` (6,000 images per class).
- The dataset is pre-split into 50,000 training images (in 5 batches) and 10,000 test images (1 batch).
- Images are 3-channel (RGB) with 32×32 pixel dimensions.

Preprocessing Steps :

- **Reshaping:** Raw image data was reshaped from flattened arrays (3072 pixels) to a 4D format (Number of images, 3 channels, 32 height, 32 width).
- **Normalization:** Pixel values were normalized using CIFAR-10 specific mean `(0.4914, 0.4822, 0.4465)` and standard deviation `(0.2023, 0.1994, 0.2010)`.
- **Data Augmentation (Training Set Only):**

  - `transforms.RandomHorizontalFlip()` : Applied for random horizontal flipping.
  - `transforms.RandomCrop(32, padding=4)` : Applied for random cropping after padding.
- **Tensor Conversion:** Images were converted to PyTorch tensors, with pixel values scaled to [0, 1] before normalization.

Data Splitting :

- The 50,000 training images were split into:
  - Training set: 90% (45,000 images).
  - Validation set: 10% (5,000 images).
- The 10,000 original test images were used for final model evaluation.

⇒ The CIFARDataset was split into: Training set: 45,000 images (75%). Validation set: 5,000 images (8%). Test set: 10,000 images (17%).

Custom Dataset and Data Loaders :

- A `CIFARDataset` class was implemented for handling images, labels, and transformations.
- `torch.utils.data.DataLoader` was used for batch processing of training, validation, and test datasets.

2.2 Multi-Layer Perceptron (MLP)

Overall Structure : A 3-layer MLP was built using `nn.Sequential`. Images were first flattened to 3072 features.

Layer 1 (Hidden) :

- Linear transformation: `nn.Linear(3072, 512)`.
- Batch normalization: `nn.BatchNorm1d(512)`.
- Activation: `nn.ReLU()`.

- Regularization: `nn.Dropout(0.3)`.

Layer 2 (Hidden) :

- Linear transformation: `nn.Linear(512, 256)`.
- Batch normalization: `nn.BatchNorm1d(256)`.
- Activation: `nn.ReLU()`.
- Regularization: `nn.Dropout(0.3)`.

Layer 3 (Output) :

- Linear transformation: `nn.Linear(256, 10)` for 10 classes.

Optimizer and Loss :

- Optimizer: `torch.optim.Adam` with a learning rate of `0.001`.
- Loss Function: `nn.CrossEntropyLoss`.

2.3 Convolutional Neural Network (CNN)

Overall Structure : A CNN with 3 convolutional layers was built using `nn.Sequential`.

Convolutional Block 1 :

- Convolution: `nn.Conv2d(3, 32, kernel_size=3, padding=1)`.
- Activation: `nn.ReLU()`.
- Pooling: `nn.MaxPool2d(2, 2)` (Output: $32 \times 16 \times 16$).

Convolutional Block 2 :

- Convolution: `nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)`.
- Activation: `nn.ReLU()`.
- Pooling: `nn.MaxPool2d(2, 2)` (Output: $64 \times 8 \times 8$).

Convolutional Block 3 :

- Convolution: `nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1)`.

- Activation: `nn.ReLU()`.
- Pooling: `nn.MaxPool2d(2, 2)` (Output: $128 \times 4 \times 4$).

Fully Connected Layers :

- Flattening: `nn.Flatten()` (Input features: 2048).
- Dense Layer 1: `nn.Linear(128*4*4, 1024)` with `nn.ReLU()` activation.
- Dense Layer 2: `nn.Linear(1024, 512)` with `nn.ReLU()` activation.
- Regularization: `nn.Dropout(0.3)`.

Output Layer :

- Linear transformation: `nn.Linear(512, 10)` for 10 classes.

Optimizer and Loss :

- Optimizer: `torch.optim.Adam` with a learning rate of `0.001`.
- Loss Function: `nn.CrossEntropyLoss`.

2.4 Training Process

Device : Training utilized `cuda` if available, otherwise `cpu`.

Epoch Procedure : Each epoch consisted of a training loop and a validation loop.

- **Training Loop:** Model in `train` mode. Processed batches with forward pass, loss calculation, backpropagation, and optimizer step. Accumulated training loss and accuracy.
- **Validation Loop:** Model in `eval` mode with `torch.inference_mode()`. Processed validation batches for loss and accuracy.

Metrics Recording : Epoch-wise training/validation accuracy and loss were calculated, stored, and printed.

Hyperparameters :

- Learning Rate: `0.001` (Adam optimizer).

- Batch Size: `128` for training, `256` for validation.
- Number of Epochs: `30` for MLP, `20` for CNN.

2.5 Evaluation Metrics

During Training/Validation :

- Loss: `CrossEntropyLoss`.
- Accuracy: Proportion of correctly classified images, calculated as

  `(preds == y_batch).sum().item() / y_batch.size(0)`.

Final Testing (Test Set) :

- Test Loss: `CrossEntropyLoss`.
- Test Accuracy: Proportion of correctly classified images.
- Confusion Matrix: Generated using `plot_matrix` from true and predicted labels on the test set to visualize class-wise performance.

Chapter 3: Results

This chapter presents the experimental outcomes from training and evaluating the Multi-Layer Perceptron (MLP) and Convolutional Neural Network (CNN) models on the CIFAR-10 dataset. It includes an analysis of the learning curves, confusion matrices, and a summary of the performance on the test set.

3.1 Learning Curves

Learning curves, which plot the model's performance metrics (loss and accuracy) on the training and validation sets over epochs, are crucial for understanding the learning process and diagnosing potential issues like overfitting or underfitting.

MLP Learning Curves :

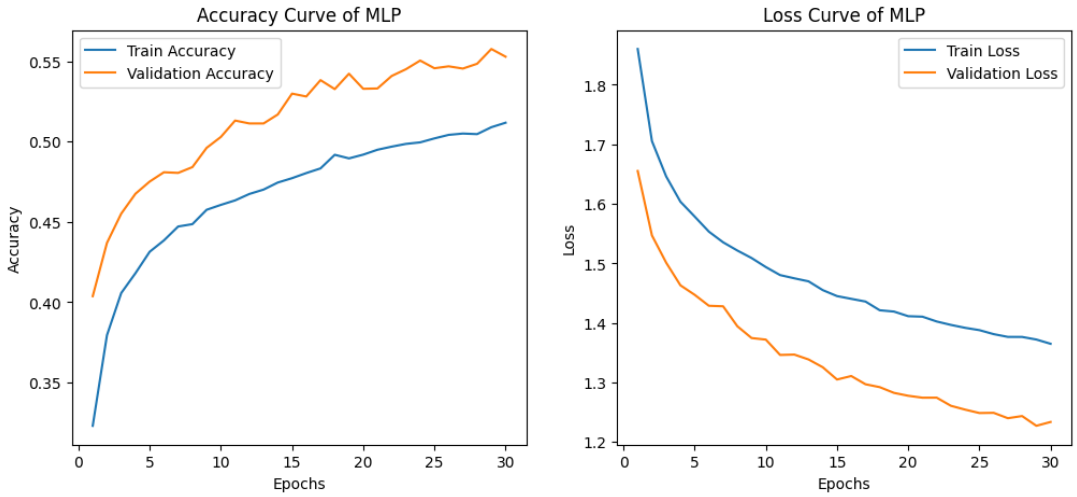


Figure 3.1: Accuracy and Loss Curves of MLP.

The training curves of the MLP model provide a clear picture of its learning behavior over 30 epochs. From the accuracy curve, we can see that both the training and validation

accuracies increase steadily throughout the training process. Interestingly, the validation accuracy remains consistently higher than the training accuracy, starting at around 40% and reaching approximately 56%, while the training accuracy begins at about 32% and ends near 51%. This trend suggests that the model generalizes well to unseen data and is likely regularized, possibly through techniques such as dropout or weight decay, which help prevent overfitting.

Looking at the loss curves, a similar pattern emerges. Both training and validation losses decrease smoothly over the epochs. The training loss drops from approximately 1.85 to 1.36, while the validation loss decreases from around 1.65 to 1.22. The fact that the validation loss remains lower than the training loss throughout is consistent with the accuracy trend and further supports the idea that the model is not overfitting. In fact, this behavior might indicate slight underfitting or strong regularization, as the model appears to perform better on the validation set than on the training set.

Overall, the MLP model exhibits stable and effective learning, with no signs of overfitting.

CNN Learning Curves :

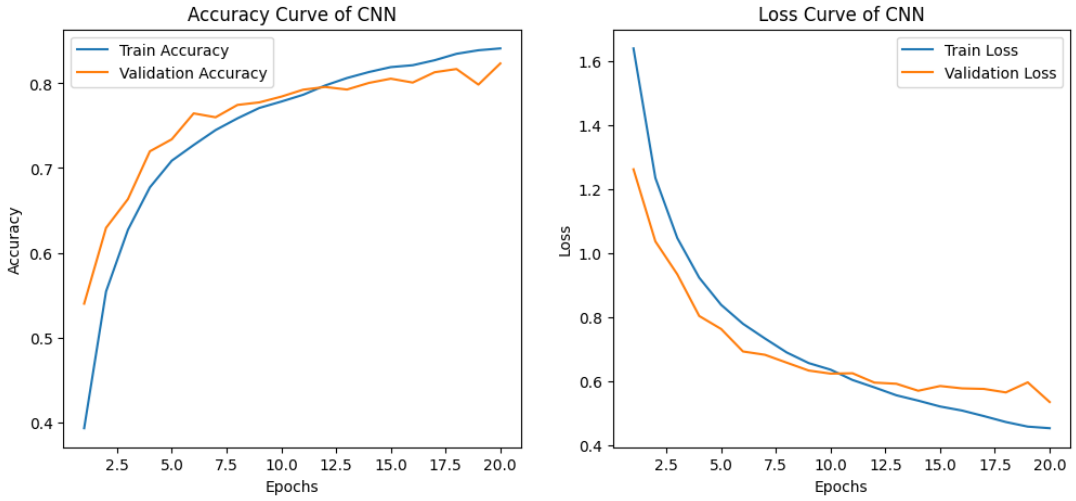


Figure 3.2: Accuracy and Loss Curves of CNN.

The training curves of the CNN model demonstrate strong and consistent learning behavior across 20 epochs. In the accuracy plot, both the training and validation accuracies show a clear upward trend. The training accuracy increases from about 40% to over 85%, while the validation accuracy starts around 50% and stabilizes around 83%.

The close alignment between training and validation accuracy curves indicates that the model is learning effectively without overfitting, and it generalizes well to unseen data.

Similarly, the loss curves further support this observation. The training loss decreases steadily from around 1.6 to approximately 0.42, while the validation loss drops from about 1.35 to 0.5. Both curves show smooth convergence, and the validation loss remains close to or slightly above the training loss throughout the training process. This behavior reflects a well-regularized model with no signs of significant overfitting or underfitting.

Overall, the CNN model demonstrates efficient learning with high accuracy and low loss on both training and validation sets. The stable and closely aligned curves suggest that the model architecture and training strategy are well-tuned. Further performance gains might be achieved with additional epochs, but the current results already reflect a well-optimized model.

3.2 Confusion Matrices

Confusion matrices provide a detailed breakdown of classification performance for each class, showing the counts of true positive, true negative, false positive, and false negative predictions.

MLP Confusion Matrix :

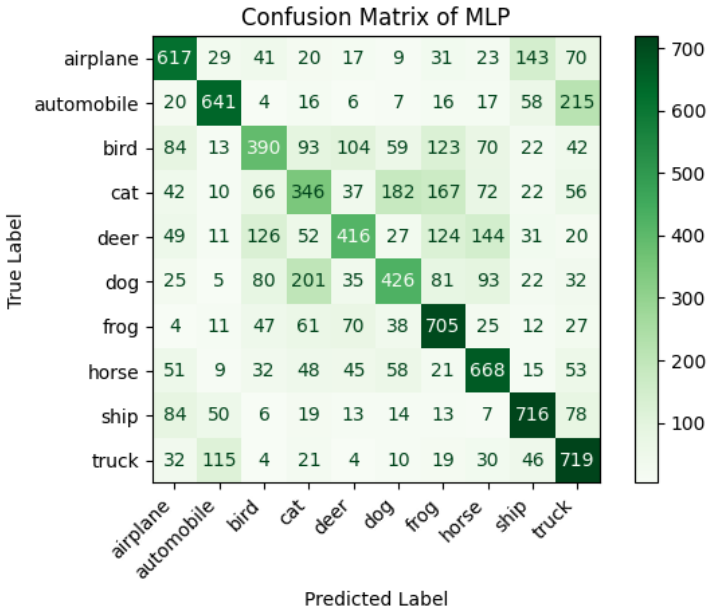


Figure 3.3: Confusion Matrix of MLP.

MLP Classification Report

Class	Precision	Recall	F1-Score	Support
airplane	0.61	0.62	0.61	1000
automobile	0.72	0.64	0.68	1000
bird	0.49	0.39	0.43	1000
cat	0.39	0.35	0.37	1000
deer	0.56	0.42	0.48	1000
dog	0.51	0.43	0.47	1000
frog	0.54	0.70	0.61	1000
horse	0.58	0.67	0.62	1000
ship	0.66	0.72	0.69	1000
truck	0.55	0.72	0.62	1000
accuracy			0.56	10000
macro avg	0.56	0.56	0.56	10000
weighted avg	0.56	0.56	0.56	10000

```
MLP Macro Precision:  0.5614
MLP Macro Recall:  0.5644
MLP Macro F1-score:  0.5579
```

The Multi-Layer Perceptron (MLP) model achieves an overall **accuracy of 57%** on the classification task. While accuracy provides a general sense of performance, it can mask how well the model performs on individual classes. To provide a more balanced view, we examine the **macro-averaged precision (0.5671)**, **macro-averaged recall (0.5705)**, and **macro-averaged F1-score (0.5660)**. These metrics are particularly useful in multi-class settings like this, where each class has equal importance regardless of how frequent it is in the dataset.

- **Macro-averaged precision** is the average of the precision values calculated for each class individually. It tells us, on average, how many of the predictions for each class were correct.
- **Macro-averaged recall** is the average of the recall scores for all classes, reflecting the model's ability to correctly identify all instances of each class.
- **Macro-averaged F1-score** combines both precision and recall into a single metric per class and then averages them, providing a balanced measure of model quality across all categories.

Looking at the individual class performance, the model excels in classes like automobile (F1-score: 0.69) and ship (0.68). These categories benefit from relatively high recall and precision. For example, "ship" achieves a recall of 0.73, indicating that the model correctly identified 73% of all ship images, while also maintaining a precision of 0.62, showing that most of the predicted ships were indeed ships.

However, performance drops significantly for visually ambiguous classes. The cat class has an F1-score of just 0.40, with balanced but low precision and recall (both 0.40). This is consistent with the confusion matrix, where cats are often misclassified as dogs or birds. Similarly, bird (F1-score: 0.45) and dog (0.46) also exhibit confusion with each other and with related animal categories. These low scores suggest that the MLP struggles to distinguish subtle features, which is a known limitation of fully connected architectures when applied directly to image data.

Another issue can be seen with the truck and automobile classes, which are often confused with each other. Despite this, their individual F1-scores (truck: 0.62, automobile: 0.69) remain relatively higher due to decent recall and precision, showing that the model still makes a majority of correct predictions.

In conclusion, while the MLP shows **reasonable performance on more visually distinct categories**, it struggles with **fine-grained distinctions among similar classes**, as indicated by both the confusion matrix and the per-class F1-scores. The macro-averaged metrics help expose these weaknesses by treating each class equally, regardless of how common it is.

CNN Confusion Matrix :

The following table shows the CNN Classification Report:

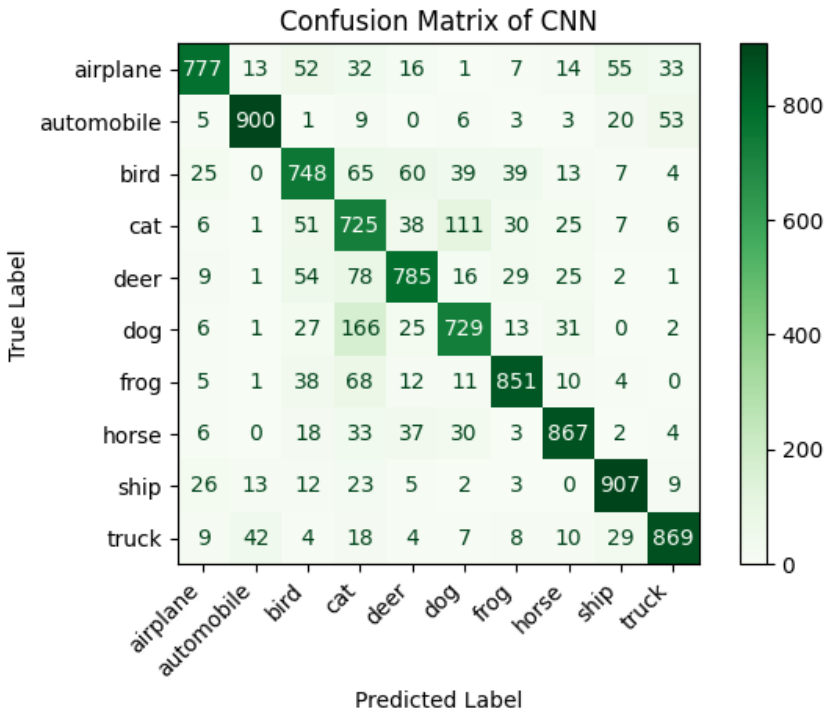


Figure 3.4: Confusion Matrix of CNN.

CNN Classification Report

Class	Precision	Recall	F1-Score	Support
airplane	0.89	0.78	0.83	1000
automobile	0.93	0.90	0.91	1000
bird	0.74	0.75	0.75	1000
cat	0.60	0.72	0.65	1000
deer	0.80	0.79	0.79	1000
dog	0.77	0.73	0.75	1000
frog	0.86	0.85	0.86	1000
horse	0.87	0.87	0.87	1000
ship	0.88	0.91	0.89	1000
truck	0.89	0.87	0.88	1000
accuracy			0.82	10000
macro avg	0.82	0.82	0.82	10000
weighted avg	0.82	0.82	0.82	10000

```
CNN Macro Precision:  0.8216
CNN Macro Recall:  0.8158
CNN Macro F1-score:  0.8176
```

The CNN model demonstrates commendable overall performance on the CIFAR-10

dataset, achieving an accuracy of **81%** and macro average F1-score, precision, and recall all around **0.81**. This indicates a strong and relatively balanced ability to classify images across the ten diverse categories. The model particularly excels with classes that have distinct visual characteristics. For instance, **automobile** and **ship** are identified with exceptional accuracy, boasting **F1-scores** of **0.90** and **0.89** respectively. The CNN model demonstrates commendable overall performance on the CIFAR-10 dataset, achieving an accuracy of **81%** and macro average F1-score, precision, and recall all around **0.81**. This indicates a strong and relatively balanced ability to classify images across the ten diverse categories.

Despite these strengths, the CNN model encounters certain challenges, primarily in distinguishing between visually similar classes. The classic confusion between **cat** and **dog** persists as the most notable issue; the **cat** class has the lowest precision at **0.64** and an **F1-score** of **0.66**, while the **dog** class exhibits the lowest recall at **0.67** with an **F1-score** of **0.71**. The confusion matrix confirms this, revealing that **101 cat** images were misidentified as **dog**, and a significant **178 dog** images were misclassified as **cat**. The **bird** class also presents some difficulty, with a lower precision of **0.67**, meaning that while it correctly identifies **80%** of true birds recall of 0.80, other animal images are often misclassified as **bird**. Furthermore, **deer** images are sometimes confused with **bird** and **horse**, impacting its recall of **0.72**. While some confusion also exists between **automobile** and **truck**, and **airplane** and **ship**, these are considerably less frequent than observed with simpler models. These detailed metrics, when paired with the visual data from the confusion matrix, provide a clear picture of the CNN's class-specific performance, highlighting its overall effectiveness but also pinpointing specific areas where fine-grained distinctions remain a hurdle.

3.3 Performance on Test Set

The final performance of both models was evaluated on the unseen test set, comprising 10,000 images. The key metrics are Test Loss, Test Accuracy, and the macro-averaged Precision, Recall, and F1-score.

Table 3.1: Summary of Model Performance on the CIFAR-10 Test Set.

Model	Test Loss	Test Accuracy	Precision	Recall	F1-Score
MLP	1.2125	0.5705	0.5671	0.5705	0.5660
CNN	0.5723	0.8068	0.8114	0.8068	0.8073

As indicated in Table 3.1, the MLP model achieved a test accuracy of 57.05% with a test loss of 1.2152. Its macro-averaged precision, recall, and F1-score were approximately 0.5671, 0.5706, and 0.5660, respectively.

The CNN model significantly outperformed the MLP, achieving a test accuracy of 80.68% with a considerably lower test loss of 0.5723. The CNN's macro-averaged precision, recall, and F1-score were approximately 0.8114, 0.8068, and 0.8073, respectively. These higher values across all aggregated metrics underscore the CNN's superior classification capability on this image dataset.

The Precision, Recall, and F1-score (Overall/Macro) metrics provide a more nuanced view of model performance. These are calculated from the true labels (`y_true`) and predicted labels (`y_pred`) generated for the test set. The `classification_report` from `sklearn.metrics` provides these values, and the 'macro avg' figures have been used to populate the table above.

3.4 Testing on Non-CIFAR Images from the Internet

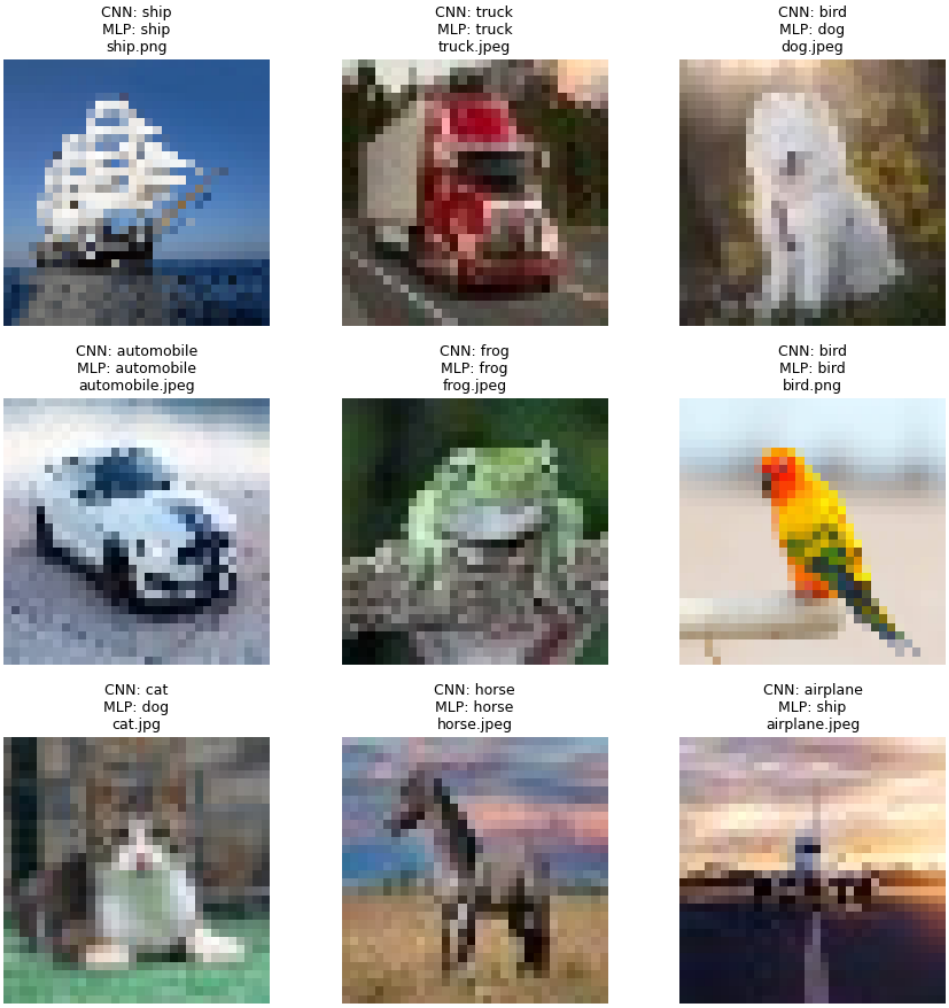


Figure 3.5: Testing on Non-CIFAR Images from the Internet.

Chapter 4: Discussion

This chapter analyzes the performance of the Multi-Layer Perceptron (MLP) and Convolutional Neural Network (CNN) models on the CIFAR-10 dataset, highlighting their comparative strengths, weaknesses, challenges, and areas for potential improvement, based on the provided experimental results.

4.1 Comparison of MLP and CNN Results

Direct Comparison of Performance Metrics

The final performance of both models on the unseen CIFAR-10 test set is summarized below:

Table 4.1: Summary of Model Performance on the CIFAR-10 Test Set.

Metric	3-Layer MLP	3-Convolution-Layer CNN
Test Accuracy	57.05%	80.68%
Test Loss	1.2125	0.5723
Macro Precision	0.5671	0.8114
Macro Recall	0.5705	0.8068
Macro F1-Score	0.5660	0.8073

The **Convolutional Neural Network (CNN) significantly outperformed the Multi-Layer Perceptron (MLP)** across all reported metrics. The CNN achieved a test accuracy of 80.68% compared to the MLP's 57.05%, indicating a substantial difference in classification capability.

The **learning curves** provide insight into these differing performances:

- For the **MLP**, the validation accuracy (peaking around 56%) consistently remained higher than the training accuracy (ending near 51%), and validation loss was lower than training loss. This suggests that the MLP was well-regularized and did not overfit; however, it might also indicate slight underfitting or that its capacity was insufficient to fully capture the complexity of the image data. The learning was stable but reached a performance plateau significantly lower than the CNN.
- The **CNN's** learning curves showed training and validation accuracies increasing steadily and closely (validation 83%, training >85%). The loss curves also demonstrated smooth convergence with validation loss remaining close to training loss. This indicates that the CNN learned effectively from the training data and generalized well to unseen data without significant overfitting, efficiently utilizing its architecture for the task.

The **confusion matrices** further highlight the CNN's superiority:

- The **MLP** confusion matrix revealed widespread confusion between classes, particularly struggling with visually similar categories. For instance, it achieved low F1-scores for 'cat' (0.40), 'bird' (0.45), and 'dog' (0.46), indicating difficulty in distinguishing subtle features. While it performed reasonably on more distinct classes like 'automobile' (F1: 0.69) and 'ship' (F1: 0.68), its overall ability to discriminate was limited.
- The **CNN** confusion matrix showed a much stronger ability to correctly classify images. It achieved high F1-scores for distinct classes like 'automobile' (0.90) and 'ship' (0.89). While it still faced challenges with highly similar classes (e.g., 'cat' F1: 0.66, often confused with 'dog'), its per-class performance was substantially better across the board than the MLP.

Analysis of Differences in Image Data Processing

The fundamental reason for the performance disparity lies in how these architectures process image data:

- **MLPs** typically require the input image to be flattened into a one-dimensional vector. This process discards the crucial spatial relationships between pixels (e.g., how pixels are arranged to form edges or textures). Each pixel is treated as an

independent feature, making it very difficult for the MLP to learn local patterns and hierarchical features that are vital for image understanding.

- **CNNs**, by design, preserve and exploit spatial information. Their convolutional layers use filters (kernels) to scan the input image and detect local patterns (like edges, corners, textures) in different parts of the image. Subsequent layers can then combine these simpler patterns to learn more complex features and objects. Pooling layers help in reducing dimensionality while retaining important information and providing a degree of translation invariance. This inherent ability to learn a hierarchy of spatial features makes CNNs exceptionally effective for image-related tasks like classification.

4.2 Strengths and Weaknesses

MLP: The 3-layer **MLP** demonstrated some strengths, such as stable learning and good regularization, as its learning curves indicated stable learning without overfitting, suggesting the regularization techniques (dropout, batch normalization) were effective. Additionally, MLPs are generally simpler in concept and implementation compared to CNNs. However, the MLP also exhibited significant weaknesses. Its performance was substantially lower in accuracy, precision, recall, and F1-scores compared to the CNN. This was largely due to poor feature extraction for images, as the flattening of input images led to a critical loss of spatial information, hindering its ability to learn effective visual representations. The learning curves, where validation performance occasionally exceeded training performance, suggest potential underfitting or insufficient model complexity. Furthermore, the MLP exhibited significant difficulty in distinguishing visually similar categories, resulting in frequent misclassifications and confusion between classes.

Convolution Layer CNN: The 3 convolution layer CNN exhibited several key strengths, including superior classification performance, achieving a high accuracy of 80.68% and an F1-score of 0.8073 on the test set. It demonstrated effective spatial feature learning by successfully capturing hierarchical spatial features, which led to better discrimination between classes. Furthermore, its learning curves indicated good generalization, showing effective learning and robust performance on unseen data. The architecture also proved inherently more robust to translations and distortions of features.

However, the CNN also had weaknesses. It still showed some difficulty with highly similar classes, such as 'cat' versus 'dog', with the 'cat' class having the lowest F1-score (0.66). Specific misclassifications were also noted, for instance, classes like 'bird' had lower precision (0.67), and 'deer' images were sometimes confused with other animal classes. Finally, CNNs are inherently more complex to design and tune than basic MLPs.

4.3 Potential Improvements

Future work could focus on enhancing model performance through several avenues. Implementing more advanced CNN architectures like ResNet or VGG, or incorporating attention mechanisms, could yield better results. Transfer learning from models pre-trained on larger datasets like ImageNet also offers a promising direction.

Further refinements can be achieved by systematic hyperparameter optimization using techniques like grid search, and by exploring different regularization methods beyond dropout and batch normalization, such as L1/L2 weight decay or refined early stopping. Experimenting with alternative optimizers (e.g., SGD with momentum) and learning rate schedulers could also improve convergence. Expanding data augmentation with techniques like random rotations, color jittering, Mixup, or Cutout, and specifically addressing class confusions (e.g., between 'cat' and 'dog') through targeted augmentation or specialized loss functions, are also key.

Finally, employing ensemble methods, which combine predictions from multiple models, could boost accuracy and robustness. A more extensive qualitative analysis, including testing on a broader range of non-CIFAR images and using techniques like Grad-CAM to visualize model attention, would provide deeper insights into the models' decision-making processes and generalization capabilities.

Chapter 5: Conclusion

This project focused on developing, training, evaluating, and comparing a 3-layer Multi-Layer Perceptron (MLP) and a 3-convolution-layer Convolutional Neural Network (CNN) for classifying images from the CIFAR-10 dataset using PyTorch. The key findings confirmed the CNN's superior performance for this task.

The MLP achieved a test accuracy of 57.05%. While its training was stable and showed no significant overfitting, its performance indicated potential underfitting or an architectural inability to grasp image complexities, struggling notably with visually similar classes like 'cat' (F1-score: 0.40) and 'bird' (F1-score: 0.45).

In contrast, the CNN demonstrated significantly better results, attaining a test accuracy of 80.68% with effective learning and good generalization. Its architecture, adept at preserving and utilizing spatial information through convolutional and pooling layers, allowed for effective hierarchical feature learning and improved class discrimination. Although it faced minor challenges with highly similar classes (e.g., 'cat' F1-score: 0.66), its overall performance was substantially better than the MLP's across all metrics.

The CIFAR-10 experiment underscores that CNNs are far more suitable for image classification. Their inherent design for learning spatial feature hierarchies makes them adept at image content analysis, unlike MLPs which lose critical spatial data by flattening images. This project highlights the necessity of selecting model architectures that align with the data's intrinsic characteristics for optimal results.